



Sunbeam Institute of Information Technology
Pune and Karad

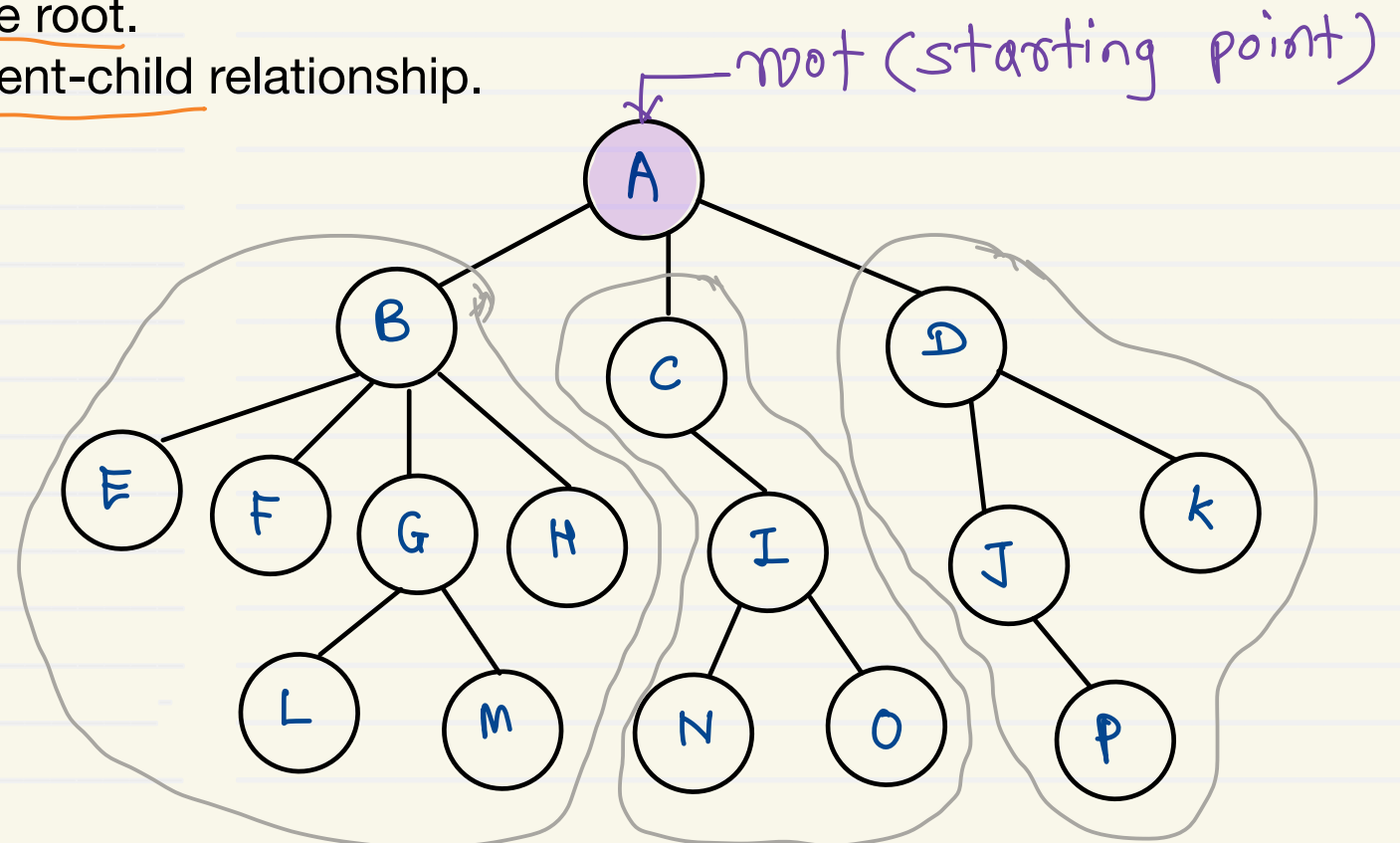
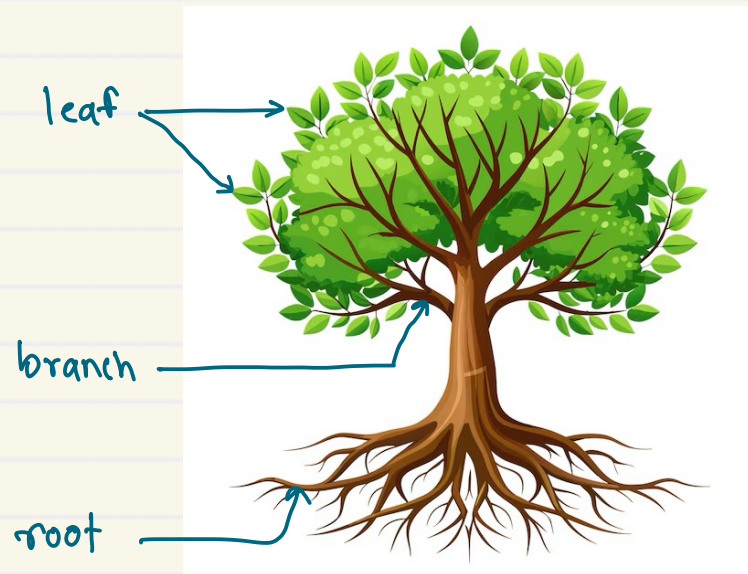
Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Tree - Definition

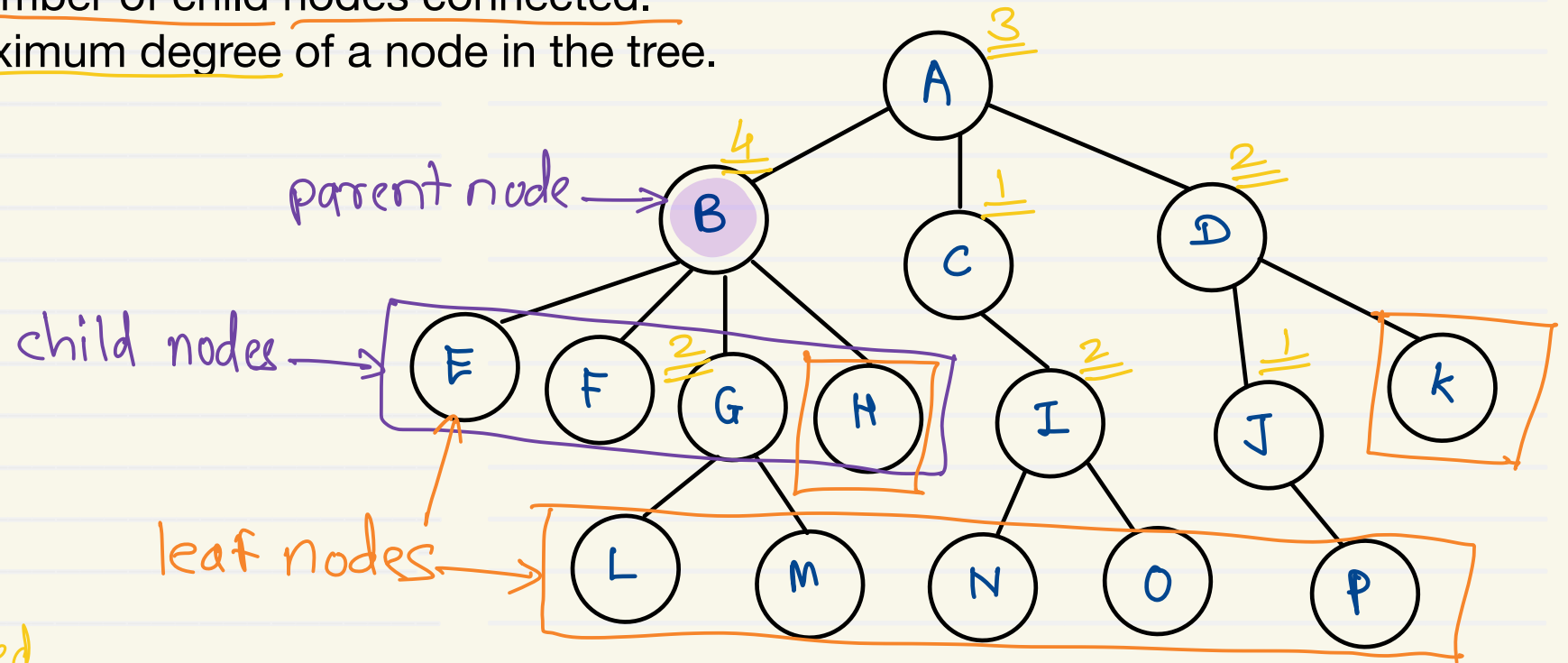
- **Tree** is a non linear data structure. – arranged in multiple levels (hierarchy)
- **Tree** is a finite set of nodes with one specially designated called as “**root**” and the remaining nodes are partitioned into m disjoint subsets where each subset is a tree.
- Each subset is called as **sub tree** of the root.
- Tree represents hierarchical data ie parent-child relationship.



Tree - Terminologies

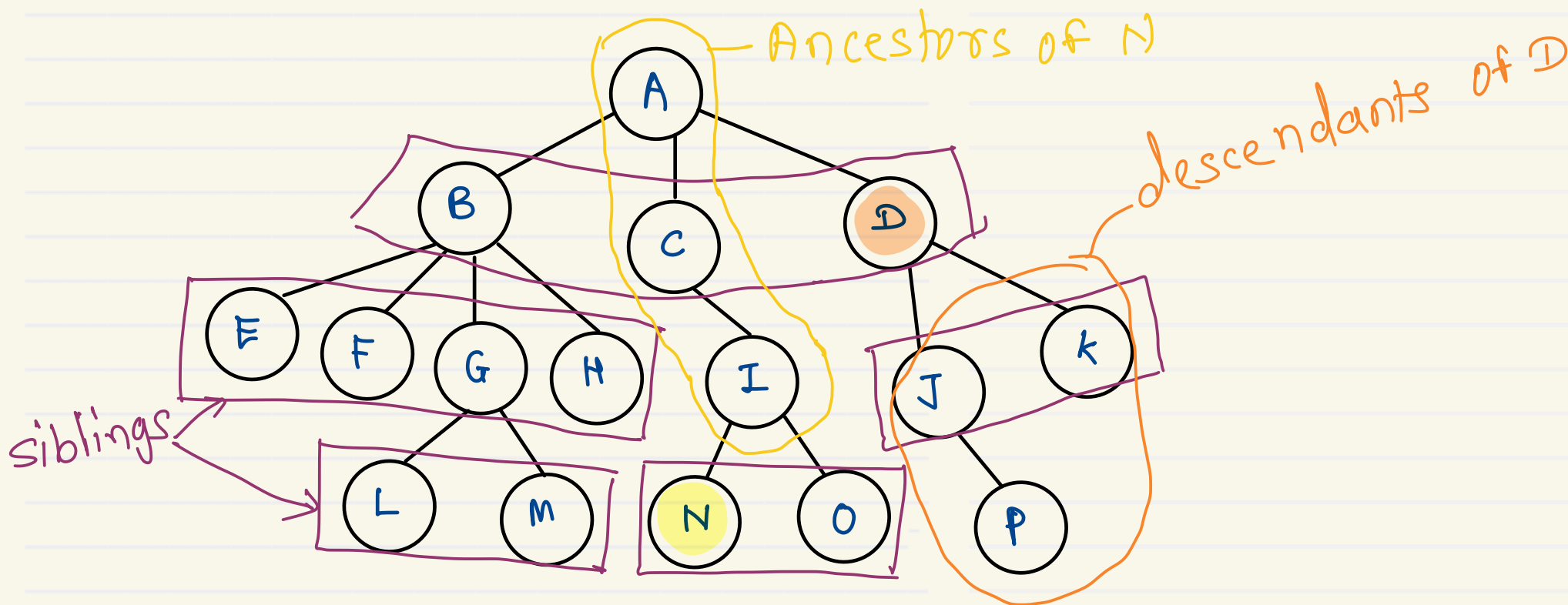
- **Node** - an item in tree that holds data and pointer to other (child) nodes.
- **Null tree** - empty tree ie tree with no nodes.
- **Parent node** - having other child nodes connected.
- **Child node** - immediate descendant of a node.
- **Leaf node** - terminal node of a tree. Does not have any child node.
- **Degree of a node** - number of child nodes connected.
- **Degree of a tree** - maximum degree of a node in the tree.

→ 4



degree (leaf node) = 0
 degree (non leaf node)
 = no. of childs connected

- **Sibling** - child nodes of the same parents.
- **Descendants** - all child nodes reachable from the node.
- **Ancestors** - all the nodes in the path from the node to the root.



Tree - Terminologies

- **Level of a node**

- Indicates the position of the node in the hierarchy
- Level of any node is level of it's parent + 1
- Level of root is 0

- **Depth of a tree**

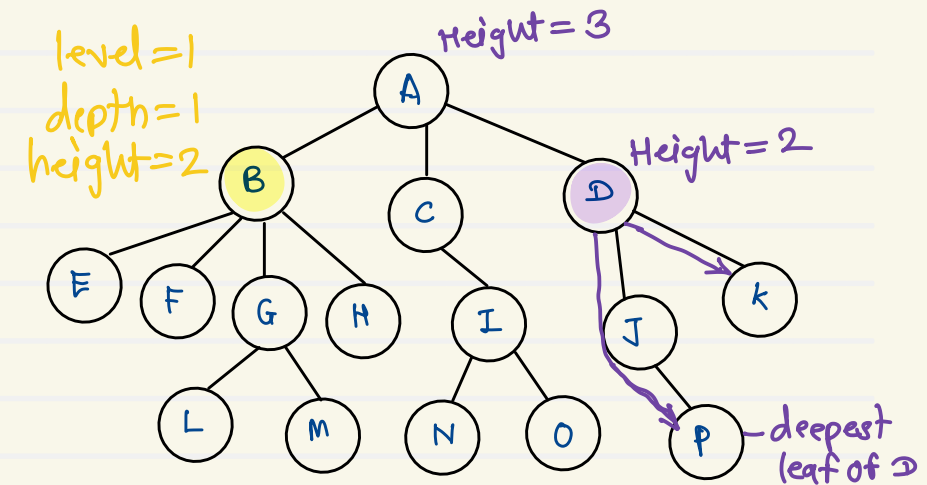
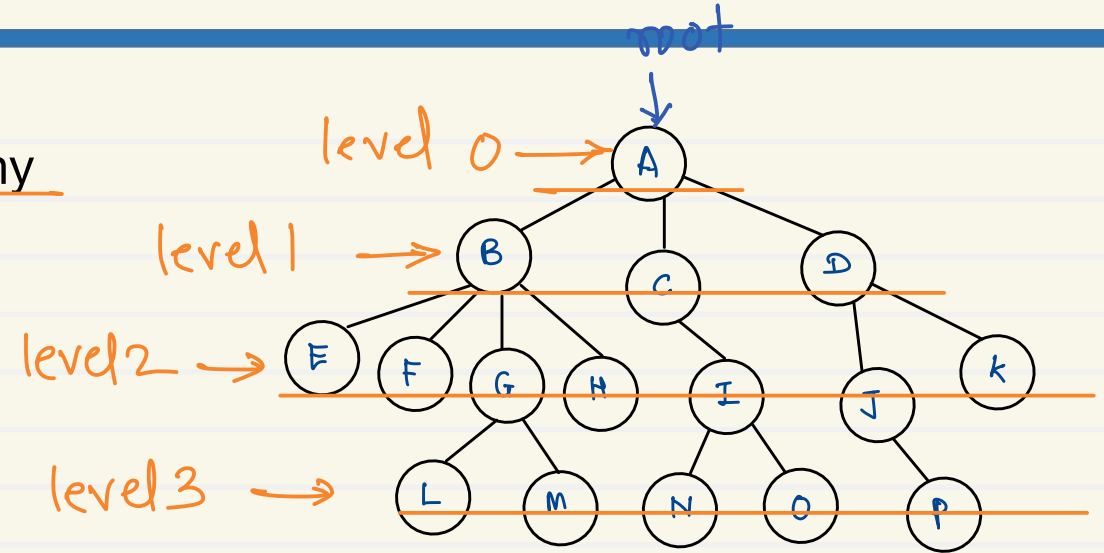
- Max level of any leaf in the tree = 3

- **Height of a node**

- Number of nodes from the node to it's deepest leaf.
- Height = 1 + MAX(left sub tree height, right sub tree height)

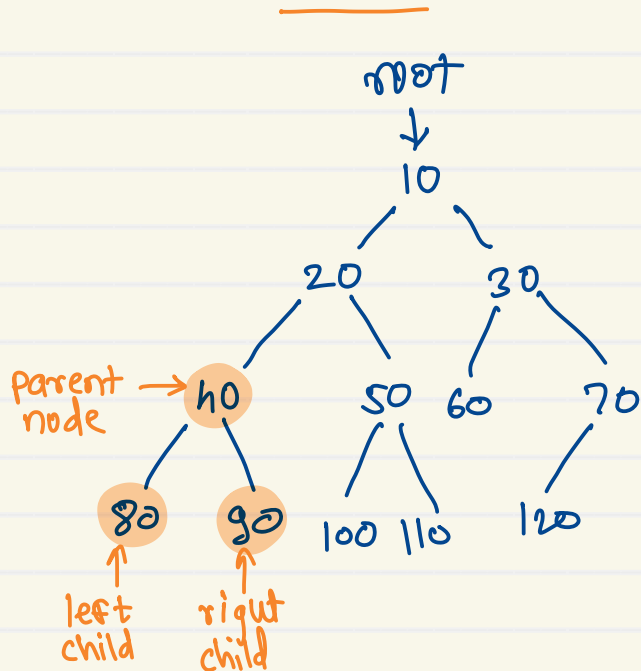
- **Height of a tree**

- Height of root of the tree = 3
- Height of null/empty tree is -1



• Binary Tree

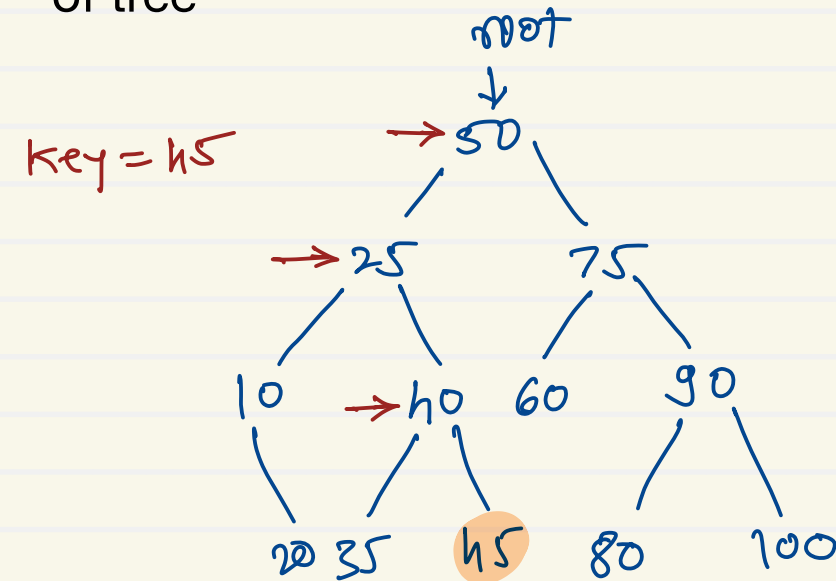
- Tree in which each node has maximum two child nodes.
- Binary tree has degree 2. Hence it is also



Binary Tree

• Binary Search Tree

- A binary tree in which left child node is always smaller and right child node is always greater or equal to the parent.
- Searching is faster.
- Time complexity : $O(h)$ h - height of tree



Binary Search Tree

Binary Search Tree - Implementation

Node :

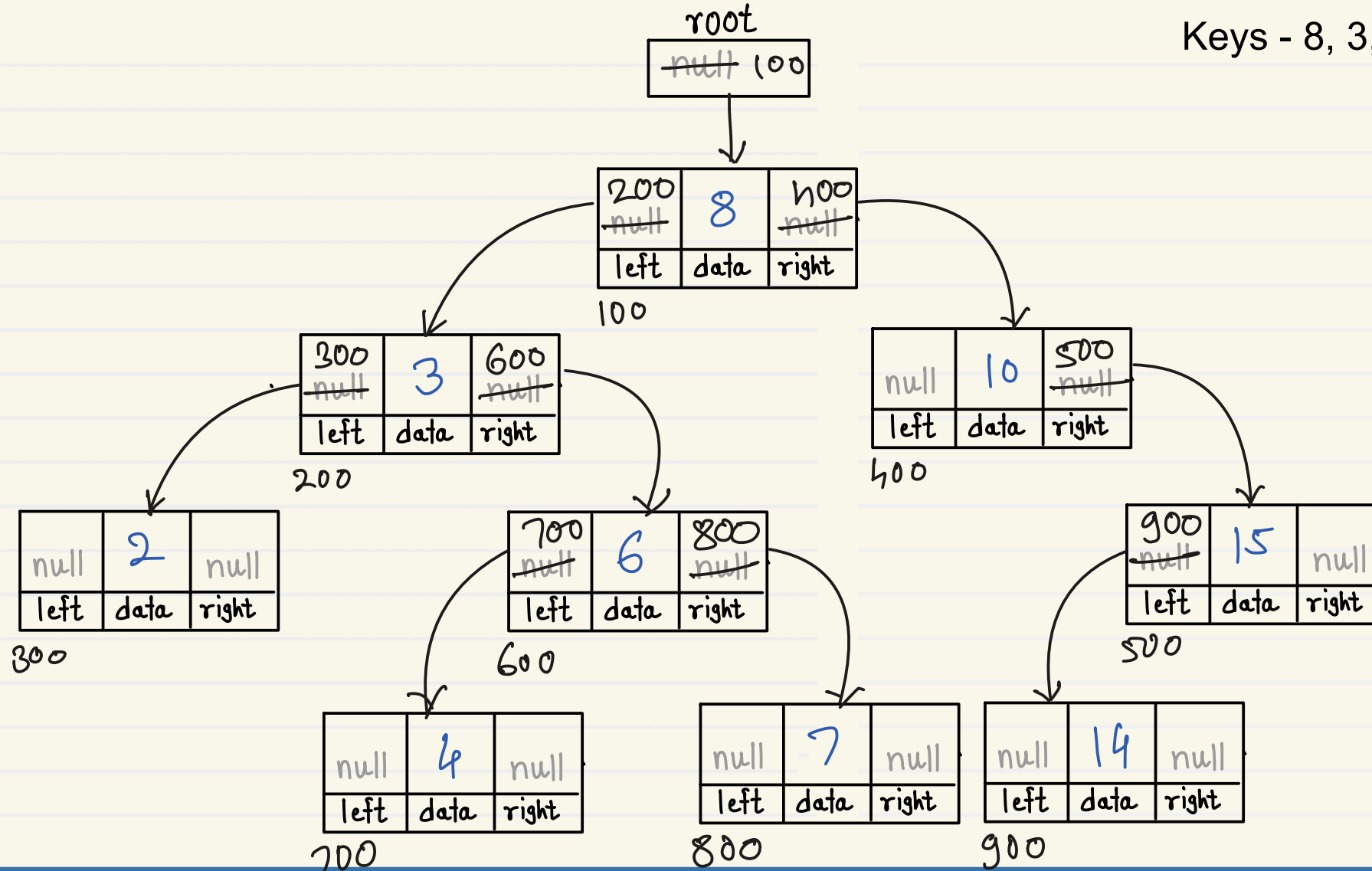
data - int, char, double, ----
left - reference/pointer to left child
right - reference/pointer to right child
height - height of a node (optional)

```
class Node {  
    int data;  
    Node left;  
    Node right;  
    ...  
}
```

```
class BST {  
    static class Node {  
        int data;  
        Node left;  
        Node right;  
        ...  
    }  
    Node root;  
    public BST() { ... }  
    public isEmpty() { ... }  
    public addNode(value) { ... }  
    public searchNode(key) { ... }  
    public deleteNode(key) { ... }  
    public display() { ... }  
}
```

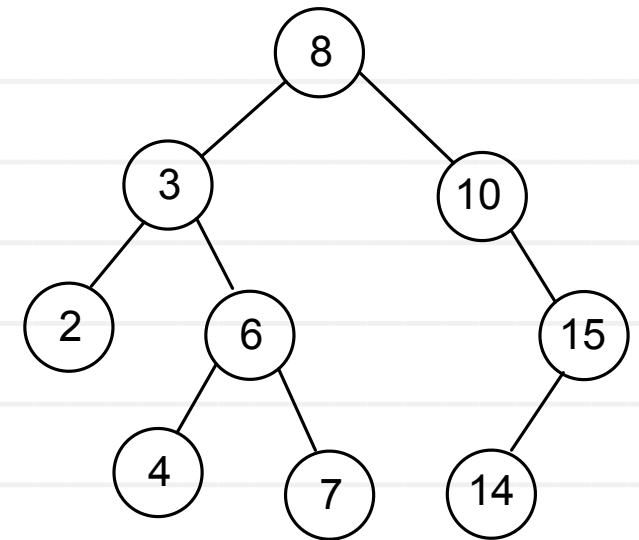
Binary Search Tree - Add node

Keys - 8, 3, 10, 2, 15, 6, 14, 4, 7, 9

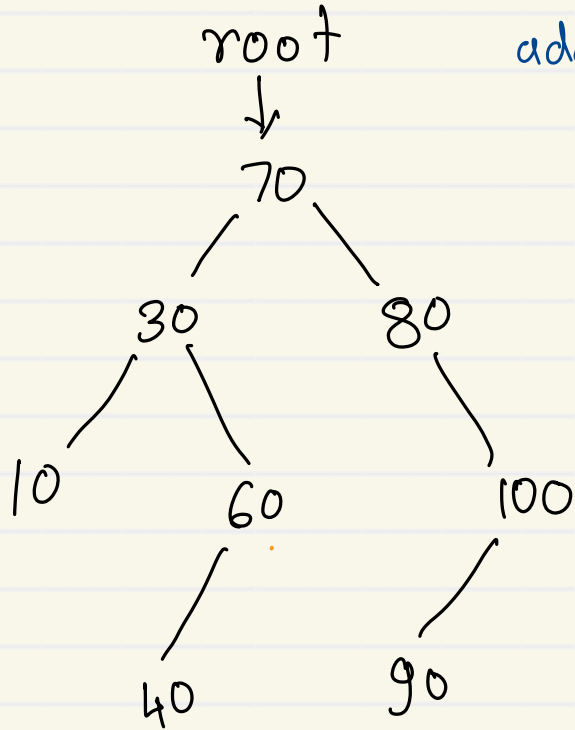


Binary Search Tree – Add Node

```
//1. create node for given value
//2. if BSTree is empty
    // add newnode into root itself
//3. if BSTree is not empty
    //3.1 create trav reference and start at root node
    //3.2 if value is less than current node data (trav.data)
        //3.2.1 if left of current node is empty
            // add newnode into left of current node
        //3.2.2 if left of current node is not empty
            // go into left of current node
    //3.3 if value is greater or equal than current node data (trav.data)
        //3.3.1 if right of current node is empty
            // add newnode into right of current node
        //3.3.2 if right of current node is not empty
            // go into right of current node
    //3.4 repeat step 3.2 and 3.3 till node is not getting added into BSTree
```



BST - Add Node Recursive



```

addNodeRec( int value) {
    if( root == null)
        root = new Node(value)
    else
        addNodeRec( root, value);
}
  
```

value = 65

```

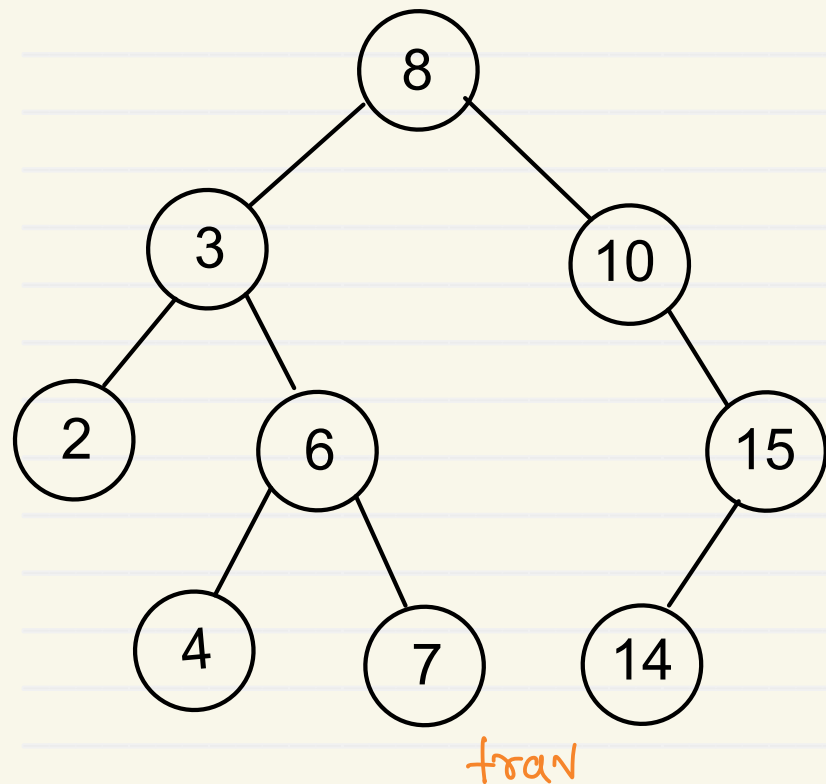
addNodeRec(&70, 65)
addNodeRec(&30, 65)
addNodeRec(&60, 65)
  
```

```

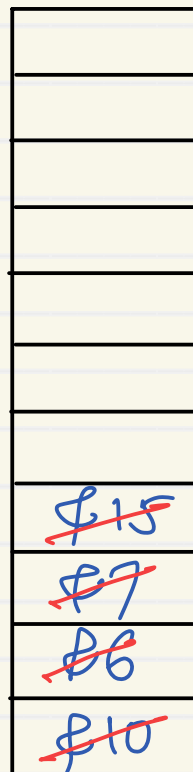
addNodeRec( Node trav, int value) {
    if( value < trav.data) {
        if( trav.left == null) {
            trav.left = new Node( value);
            return;
        } else {
            addNodeRec( trav.left, value);
        }
    } else {
        if( trav.right == null) {
            trav.right = new Node( value);
            return;
        } else {
            addNodeRec( trav.right, value);
        }
    }
}
  
```

BST - Preorder (Non recursive)

VLR



Stack

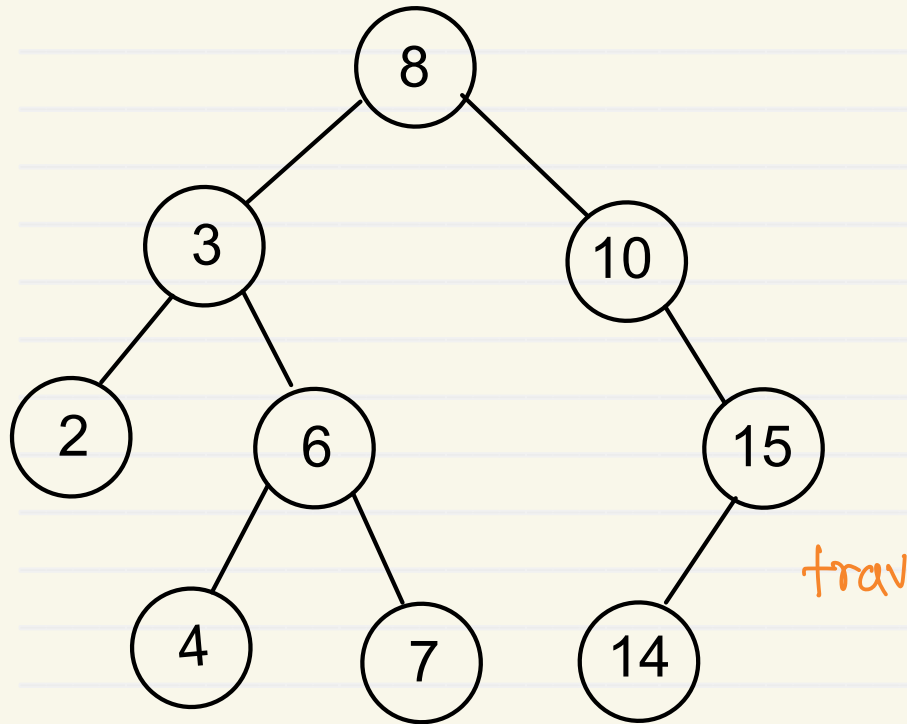


1. create stack to push nodes
2. start traversing from root node
3. visit current node
4. push right on stack if exists
5. go on left node
6. repeat step 3 to 5 till extreme left (trav = null)
7. pop node from stack.
8. repeat step 3 to 7 untill stack becomes empty or trav becomes null

Preorder : 8, 3, 2, 6, 4, 7, 10, 15, 14

BST - Inorder (Non recursive)

LVR



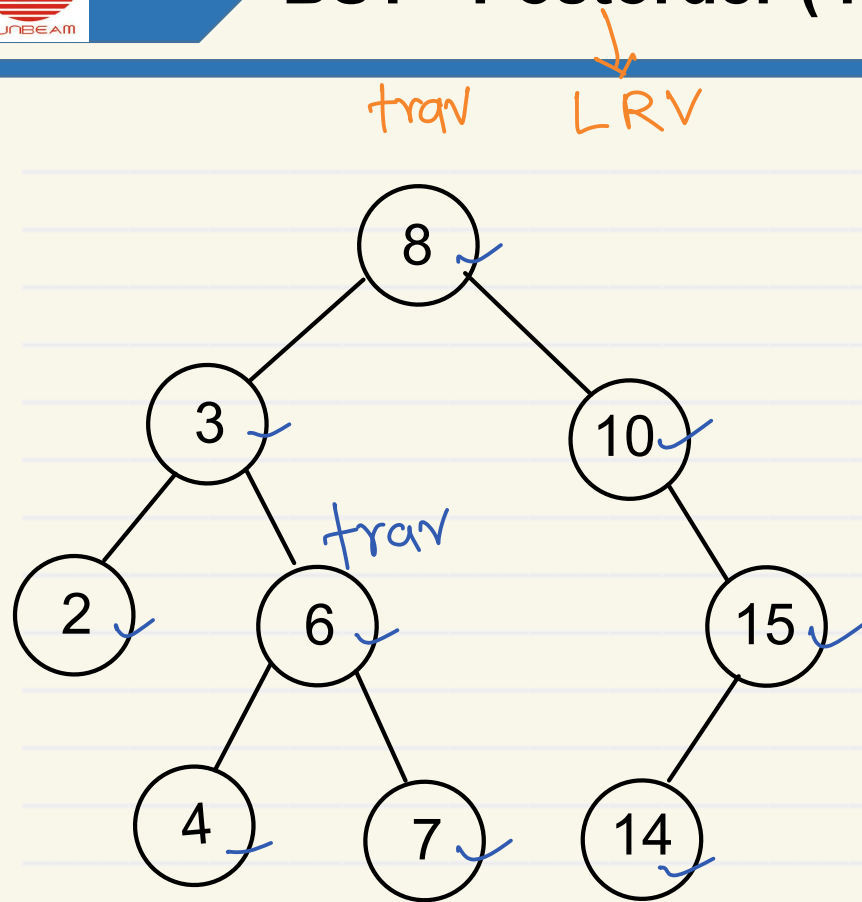
Stack

14
15
10
7
4
6
2
3
8

1. create stack to push nodes
2. start traversing from root node
3. push current node on stack
4. go on left node
5. repeat step 3 and 4 till extreme left (trav = null)
6. pop node from stack
7. visit current node
8. go on right side node
9. repeat step 3 to 8 until stack is empty or trav is null.

Inorder : 2, 3, 4, 6, 7, 8, 10, 14, 15

BST - Postorder (Non recursive)



Stack

14
15
10
10
8
7
6
4
6
3
2
3
8

1. create stack to push nodes
2. start traversing from root node
3. push current node on stack
4. go on left node
5. repeat 3 & 4 till extreme left
6. pop node from stack
7. if dont have right or right is visited
8. visit popped node & mark it as visited
9. if has right & not visited
10. push current node
11. go on right side node
12. repeat 3 to 11 till stack is not empty or trav is not null.

Postorder : 2, 4, 7, 6, 3, 14, 15, 10, 8